Milestone 5 Beta build and presentation preparation

I will first construct the <theory-of-the-program>, then generate <program text> only after the theory is explicit.

OPERATIONAL_DIAGNOSTIC

Substantives_Demolished Replace vague uses of polish, improvement and readiness with the actions and records below.

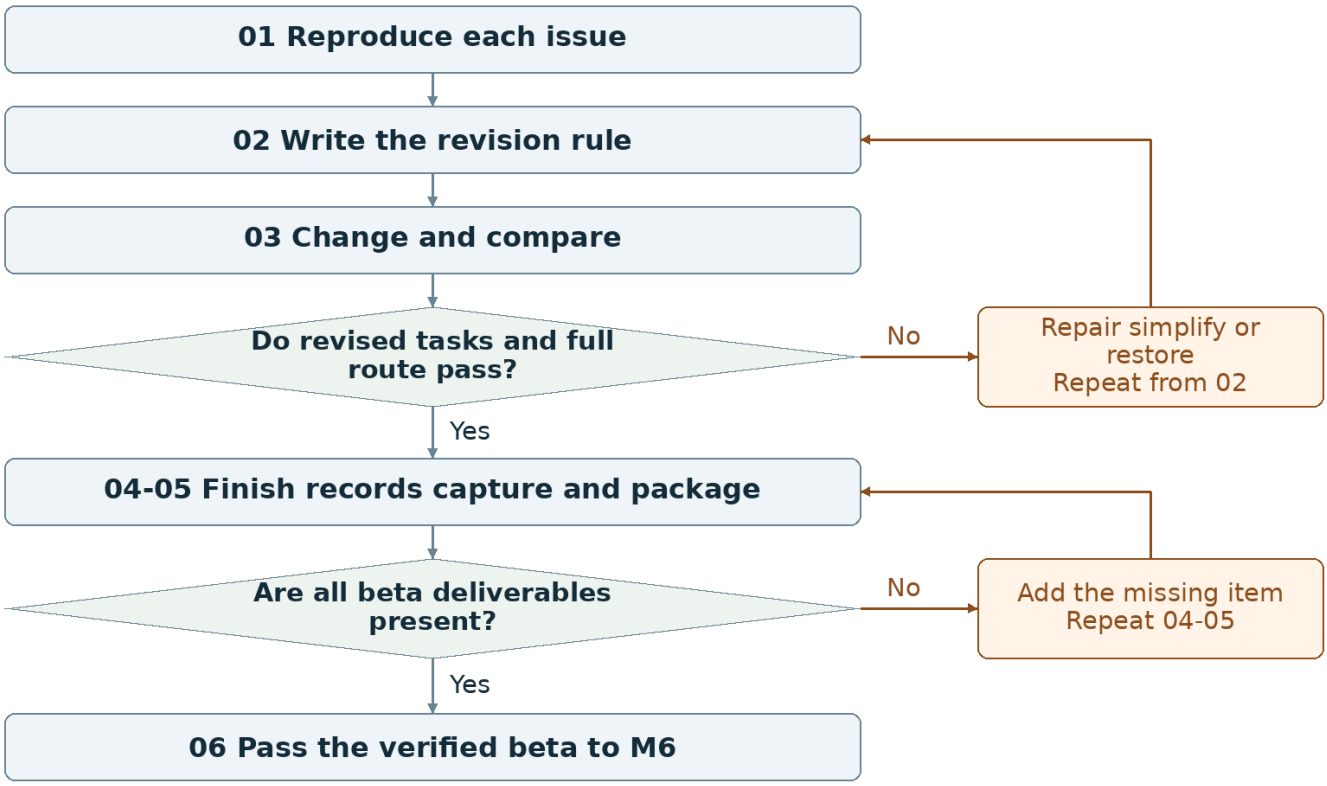
Occult_States_Removed Replace the interface feels better with the changed label or response and the result of replaying the observed task.

The_Language_Game Make one bounded revision, repeat its check, and preserve the full route before moving to presentation work.

CONTROL_SURFACE_DRAFT

Apply the selected revisions, repeat the affected actions, and prepare the beta title, designer statement and four screenshots.

Start with Open the alpha, two peer notes, top-three revision plan, current rules and tested-package record. Keep a copy of the prior working build.



Use the numbered procedure; repeat checks only where the repair affects their result.

Required package Submit the beta and required records in the current Canvas fields; use a named folder or ZIP only as a packaging convention.

Assign maker _____ and checker _____. Swap roles at the next task. Leave checks pending until performed.

Milestone 5 Write the rules

1 <Initial Interpretation>

For each selected revision, <maker> [repeats] <recorded input>; <checker> [records] <displayed response>; <maker> [edits] <named code or asset>; <checker> [repeats] <input> and [compares] <observed response> with <expected response>. Preserve the chosen player task and ending unless the group records a rule change.

2 <Theory Skeleton>

Record	What to write
<Entities>	Track each issue, observation, revision, expected result, actual result and build version.
[Operations]	Reproduce the problem, change a bounded part, rerun the task, compare results and keep or restore the change.
<States>	List states affected by the revision and all states on the complete route. Check that an interface change has not hidden a required transition.
<Conditions>	Record the old and revised input or outcome condition. Keep verification status in the revision record; reopen M2 when a core rule changes.
<Invariants>	For each adopted change, check scoped rules retained from M2, including valid value ranges. Record expected reset values as postconditions of reset.
<Constraints>	Retain required interaction and outcomes. Supply final title, short designer statement, four actual screenshots and draft credits.

Use conditions to decide whether an action or transition is allowed. State the scope of each invariant and how to check it. Keep submission requirements separate from runtime rules.

3 <Assumption Ledger>

Status	Action
<safe>	Enter an unverified premise only with its basis and limited consequence if wrong. Keep alpha observations as evidence beside proposed revisions.
<uncertain>	Treat the effect of each proposed revision and any new presentation setup as untested.
<requires-user-decision>	Record adopted fixes, removed optional features and deferred work with reasons. Do not defer required behavior and still mark the stage passed.

Use {issue, evidence, planned change, owner, expected output, observed output, disposition}. Keep a proposed improvement separate from a verified result.

Record the next open item Claim _____ Status _____
Owner _____ Evidence/check _____ Next action _____

Write <entity> for a person, object, value or state; [operation] for an action; and {record} for related fields. Record planned, observed, failed or not tested beside each claim. Do not mark a check passed until a person performs it and records the result.

Milestone 5 Run the procedure

4 <Operational Description>

Read each action aloud before doing it. The maker performs the action; the checker records the named result. Use the return instruction when a check fails.

Step	Perform and save	Check and return
01 Reproduce each issue	Use the alpha's recorded inputs and note the current result. Rewrite a vague issue as an observable action and response before editing.	If the problem cannot be reproduced, record that limit and collect enough detail to make a bounded change.
02 Write the revision rule	State the expected response and the affected state. Mark whether the change alters a game rule, control or display. Update the systems plan first when rules change.	Both partners predict the same post-change result before implementation.
03 Change and compare	Implement one revision at a time. Repeat the recorded task and compare expected versus actual behavior. After consequential changes, recheck the complete beginning-to-ending route.	Keep verified changes. Repair or restore a revision that introduces a failure; repeat affected checks.
04 Finish the beta records	Record each priority as verified, simplified/removed or deferred with a reason. Choose the final title. Write a short designer statement naming the action, response and film connection present in the beta.	Remove unsupported claims. Leave substantial unfinished core work in the active task list instead of labeling it presentation polish.
05 Capture and package	Capture four distinct screenshots from the beta. Update draft credits/references, progress note and AI statement. Rehearse the actual beta launch and route.	Match all four images and claims to the same build. Use actual screenshots, not unimplemented mockups.
06 Hand off	Pass the checked beta, title, statement, screenshots, revision results and remaining limitations to M6.	List only final presentation tasks after required behavior and records pass their checks.

5 <Failure Description>

Observed problem	Required response
A fix changes another state unexpectedly	Restore the working behavior or repair the affected rule in step 03; rerun that route.
Optional feature adds an untested dependency	Remove or simplify it, then repeat the recipient launch and route check.
Four images show an older version	Recapture the beta in step 05; label the build and visible screen/state.

Use the map A no branch returns to the named task. Repair the recorded problem and repeat that check; do not mark the stage passed because the planned time window ended.

Milestone 5 Test changes and produce the work

6 <Change Test>

Ask: "What would change if the requirements changed?" The maker writes the expected result first. The checker performs or traces the change and records the result. Swap roles for the second scenario.

Modification	Run this check
Remove a confusing control	In a test copy, remove or simplify the optional control. Repeat the observed task and verify that required actions and status information remain reachable. Keep or reject the change from the recorded result.
Mute sound	Run the route with sound muted. For each essential sound cue, check that visible information also communicates the needed state or action. Add that display if it is absent and repeat the task.

Save {version or design record, starting condition, input/change, expected result, observed result, pass/fail/not tested, next action}. Use paper traces in M1-M2; report executed checks only when they occurred.

7 <Implementation Plan>

Give each revision an owner and a different checker. Keep changes small enough to associate a result with one edit. Capture screenshots after the checks, and update credits while adding or replacing assets.

8 <Program Text>

Implement only adopted revisions and bounded defect repairs. Update rules, instructions and comments together when behavior changes. Use targeted AI help only for a named problem; record what was kept, changed or rejected.

9 <Theory-Code Mapping>

Element	Record the connection
<types>	Name data added, removed or reinterpreted by each revision.
<functions>	List changed handlers or rendering functions beside their issue IDs.
<classes>	Keep existing responsibilities traceable; avoid restructuring without a stated need.
<tests>	Attach before/after task results and the complete-route check.
<comments>	Update outdated rules and AI assistance comments.
<configuration>	Record final beta values and the evidence for adjustments.

Use actual file and function names when code exists. Label planned counterparts proposed; write not applicable when a structure is absent.

Carry forward the course rules

Keep the project connected to WarGames. Build in Processing using p5.js mode; accept interactive input; include at least three distinct screens or states; provide a start and reachable outcome; credit external material; organize and comment the code.

Record actual AI use at every milestone, including use of this manual if adopted. Use AI for bounded brainstorming, explanation, small examples you study and adapt, specific debugging or review. Author the project as a group; do not request an entire generated project. Identify assisted code and your changes in comments.

Milestone 5 Submit and hand off

10 <Residual Human Theory>

Record why the group kept, simplified or rejected each proposal. Each partner identifies one verified change and shows the before/after action result. Keep unresolved limitations beside the build that still contains them.

Check the required submission

- A beta build.
- The final title and a short designer statement.
- Four actual screenshots from the beta.
- Draft credits and references.
- Short progress note and AI Use Statement.

Use clearly named files or a folder/ZIP if the Canvas fields permit it. The supplied instructions do not specify a container or minimum PDF length for M5.

Completion check The checker repeats the adopted revision checks and the complete route without a required behavior failing. The final title, statement, four screenshots and records all match the beta.

Progress note Name the current version; state what was completed and checked, what is missing, and the next action with its owner. Add the screenshot/capture supporting the current status.

Complete the AI Use Statement

Field	Enter the actual record
Tool and dates	Name the tool, or record no AI used only if accurate. Enter actual dates of use.
Request and affected work	Summarize the request. Name the text, image, planning record or code file affected.
Kept changed rejected	Record the output retained, edits or rewrites made, and material discarded.
Group confirmation	Recommended check: each partner traces actual submitted code; for planning-only work, trace the proposed interaction. Identify image sources, retained edits and displayed information. Then complete the required group confirmation covering code and images; leave it pending until both partners can confirm it.

If this AI-produced manual is adopted, record ChatGPT, October 21 2026, milestone planning and operational rewrite, and your actual kept/changed/rejected choices. Do not copy a completed student confirmation from this manual.

Handoff Pass one identified beta and its evidence to M6. Record any optional deferred work so it is not accidentally promised during the presentation.

Record: maker _____ checker _____ version _____
Checks still pending _____ submitted file(s) _____ upload status _____

Requirements source: supplied WarGames project brief and milestone instructions. Treat roles, file names, internal checks and relative-week allocations as recommended working procedures. Use corrected deadlines from the course; none are asserted here.